

Transaction Pathway

1) Find students who achieved 'A' in ALL their courses

Transaction Start → DBMS receives SELECT request → Scheduler checks concurrency → Shared Locks assigned on: Student, Enrollment → DBMS calls Student table (PK = id) → DBMS calls Enrollment twice:

first subquery: all courseCodes of the student

second subquery: courseCodes where grade = 'A' → Execute EXCEPT condition → Apply NOT EXISTS filter → Return matching students → Release Shared Locks → Log read entry

2) Courses scheduled in rooms with max capacity

Transaction Start → SELECT received → Shared Locks assigned on: Course, Room, Schedule → Call Schedule using FK = roomNum + courseCode → Call Room: capacity attribute used → Call Course using PK = code → Compare room capacity with ALL other rooms for same course → Apply EXISTS logic → Return course names → Release locks → Log entry → Transaction End

3) Students enrolled in more than two courses

Transaction Start → SELECT received → Shared Locks on: Student, Enrollment → Call Student table → Call Enrollment using FK stdID → Group by student ID → Apply HAVING count(courseCode) > 2 → Return students → Release locks → Log → Transaction End

4) List instructors whose total teaching credit load exceeds the university average

Transaction Start → DBMS receives SELECT → Shared Locks on: Instructor, Schedule, Course → Call Schedule using FK instID + courseCode → Call Course using PK code to get credits → Group teaching load per instructor → Inner subquery: compute credit totals per instructor → Outer query: compare sum(credits) > average load → Return matching instructors → Release locks → Log → Transaction End

5) Students whose contact numbers are guaranteed unique

Transaction Start → SELECT received → Shared Locks on: Student, Std_contact → Call Student → Call Std_contact using FK = studentID → Group by student → Apply HAVING: count(contact) = count(distinct(contact)) → Return students → Release locks → Log → Transaction End

6) Majors where ALL students have GPA > 3.00

Transaction Start → SELECT received → Shared Locks on: Major, Student → Call Major using PK = id → Subquery calls Student using FK majorID → Check NOT EXISTS student with GPA ≤ 3.00 → Return majors that satisfy the rule → Release locks → Log → Transaction End

7) Students enrolled in ALL courses taught by instructor "Zewail"

Transaction Start → DBMS receives SELECT → Shared Locks assigned on: Student, Instructor, Schedule, Enrollment → Call Instructor, filter Iname = 'Zewail' → Call Schedule to retrieve courses taught by him → Inner EXCEPT: courses taught EXCEPT courses taken by student → Apply NOT EXISTS → Return students → Release locks → Log → Transaction END

8) Display full schedule: course + professor + day + time"

Transaction Start → DBMS gets SELECT → Scheduler checks → Assign Shared Locks on: Schedule, Course, Instructor → Call Schedule (PK = sched_id) → Call Course using FK courseCode → Call Instructor using FK instID → Perform RIGHT + LEFT joins → Return full schedule → Release locks → Log → Transaction End